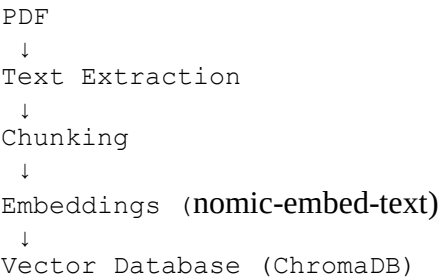


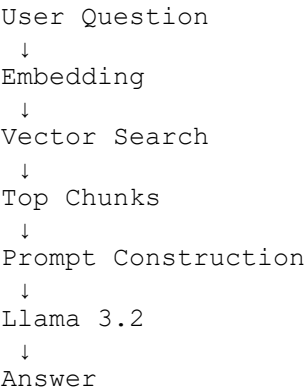
RAG Systems Tutorial

End-to-End Flow

DOCUMENT INGESTION



QUESTION ANSWERING



Important Components In Your Project

Component	Purpose
FastAPI	Backend APIs
React	Frontend
PyMuPDF	PDF text extraction
Chunking Service	Split document
nomic-embed-text	Generate embeddings
ChromaDB	Store vectors
Similarity Search	Find relevant chunks
Llama 3.2:3b	Generate answers
RAG Service	Orchestrates retrieval + generation

Common Interview Question

What is the difference between an LLM and a RAG system?

LLM

Question

↓

Model

↓

Answer

Knowledge limited to training data. May hallucinate.

RAG

Question

↓

Retrieve Knowledge

↓

Provide Context

↓

Generate Answer

Uses external knowledge. More accurate.

Can answer questions about documents uploaded 5 minutes ago.

The Three Most Important Concepts

If you remember only three things about RAG, remember these:

1. Embeddings

Text → Vector

Represent meaning.

Embedding models convert text into numbers. Suppose we have -- "I love dogs" --- Humans understand the meaning but computers don't. Embedding model converts it into a long list of numbers like [0.123, -0.452, 0.887, ...]. This list is called a vector embedding.

What is the heart of a RAG system?

Embeddings and vector search. The LLM generates the final answer, but embeddings are what allow the system to find relevant information.

***Why Embeddings Matter? Why Embeddings Are Critical?

Without embeddings:

No vector search
No semantic search
No RAG

They are the foundation of retrieval.

- Consider sentence A:
How many leaves are allowed?

sentence B
What is the vacation policy?

Different words but similar meaning. *Keyword search struggles here. Embedding search succeeds because similar meanings produce vectors that are close together.

2. Retrieval

Vector Search

Find relevant information.

Why Retrieval Comes Before Generation?

Many newcomers think, this is RAG:

Question

↓

LLM

↓

Answer

But it isn't RAG.

RAG is:

Question

↓

Retrieve Knowledge

↓

Provide Context



LLM Answer

Without retrieval - Just Chatbot

With retrieval - RAG System.

Retrieval Pipeline

PDF



Text Extraction



Chunking



Embeddings



ChromaDB Storage



Semantic Retrieval

3. Augmentation

Inject retrieved information into prompt

before calling the LLM.

Everything else in RAG exists to support these three steps.

What Is "Augmented" In RAG?

This is a common AI interview question.

Suppose the document contains --- Employees receive 20 annual leaves.

User asks --- What is the leave policy?

We retrieve --- Employees receive 20 annual leaves.

Then we augment the prompt:

Context: Employees receive 20 annual leaves.

Question: What is the leave policy?

This augmented prompt is sent to Llama. That is why it's called --- Retrieval-Augmented Generation.

Complete RAG Flow (Document Q&A System)

Phase A: Document Ingestion (One-Time Process)

This happens when a user uploads a document.

```
PDF Upload
  ↓
Text Extraction
  ↓
Chunking
  ↓
Embedding Generation
  ↓
Store in Vector Database
```

1. PDF Upload

User uploads:

Employee_Handbook.pdf

Backend receives and saves it.

2. Text Extraction

PDF:

```
Page 1
Page 2
Page 3
```

becomes:

Welcome to the company...

Employees receive 20 annual leaves...

Working hours are 9 AM to 6 PM...

Purpose:

PDF → Plain Text

3. Chunking

Large text is split into smaller pieces.

Example:

Chunk 1

Welcome to the company...

Chunk 2

Employees receive 20 annual leaves...

Chunk 3

Working hours are 9 AM to 6 PM...

Purpose:

- Fit into context window
 - Improve retrieval accuracy
 - Reduce search space
-

4. Embedding Generation

Each chunk becomes a vector.

Chunk 1

↓

Vector A

Chunk 2

↓

Vector B

Chunk 3

↓

Vector C

Using:

openai.embeddings

Purpose:

Text → Meaning Representation

5. Store In Vector Database

Store:

Chunk Text
+
Vector
+
Metadata

Example:

ID: 101

Text:
Employees receive 20 annual leaves.

Vector:
[0.24, -1.83, 0.56 ...]

Metadata:
Page 12
Document: Employee Handbook

Stored in:

ChromaDB

Phase B: Retrieval (When User Asks A Question)

This happens every time the user asks something.

Question
↓
Embedding
↓
Vector Search
↓
Top Relevant Chunks

6. User Question

Example:

What is the vacation policy?

7. Question Embedding

Question becomes:

Vector Q

Using same embedding model:

```
nomic-embed-text
```

Important:

Document chunks and user questions must use the same embedding model.

8. Similarity Search

ChromaDB compares:

```
Question Vector
```

```
with
```

```
Chunk Vector A
```

```
Chunk Vector B
```

```
Chunk Vector C
```

Example:

```
Distance(A) = 0.91
```

```
Distance(B) = 0.12
```

```
Distance(C) = 1.45
```

Smallest distance wins.

Retrieved:

```
Employees receive 20 annual leaves.
```

9. Top K Retrieval

Usually retrieve:

```
Top 3
```

```
Top 5
```

```
Top 10
```

chunks.

Example:

```
Chunk 2
```

```
Chunk 7
```

```
Chunk 15
```

Purpose:

Provide enough context.

Phase C: Augmentation

Now we build the prompt.

Retrieved Chunks

↓

Prompt Construction

10. Context Construction

Retrieved chunks:

Employees receive 20 annual leaves.

Leave requests require manager approval.

Unused leave cannot be carried forward.

Combined into:

Context:

Employees receive 20 annual leaves.

Leave requests require manager approval.

Unused leave cannot be carried forward.

11. Prompt Building

Prompt becomes:

You are a document assistant.

Answer only using the provided context.

Context:

Employees receive 20 annual leaves.

Leave requests require manager approval.

Question:

What is the vacation policy?

This is the "A" in RAG:

Augmented

because we augment the prompt with retrieved knowledge.

Phase D: Generation

Now the LLM is involved.

Augmented Prompt

↓

LLM

↓

Answer

12. LLM Processing

Using:

Llama 3.2

The model reads:

- Question
- Retrieved context

and generates answer.

13. Final Answer

Example:

Employees receive 20 annual leaves. Leave requests require manager approval.

Returned to user.